

Building a Narrative from Data

WEEK 2

FINS5545: FINANCIAL MARKET DATA LITERACY

LEARNING OBJECTIVES

By the end of this lecture, you will be able to:

- Describe the four **standards** a publication-quality figure should meet — caption, units, source, sample window
- Read Reserve Bank of Australia (RBA) and Federal Reserve Economic Data (FRED) tables into pandas directly from the web, and convert the date column to `datetime64` (the numbers usually arrive as `float64` already)
- Apply Stage 1 of the Data Factory Floor (DFF) to an RBA dataset — download, parse, type, date-map, error-check, lag-check, freeze
- Resample daily data to month-end and merge it with monthly macroeconomic data
- Choose the right transformation — level, first difference, log growth, or return — for the question at hand
- Compare six figure types and justify the figure choice for a given dataset and narrative
- Build a five-figure visual narrative of the Australian macroeconomy

REQUIRED READING

Course-mandated textbook

Hric and Lin (2026). *Applied Data Science in FinTech: Models, Tools, and Case Studies*, **Chapter 3 (“Data Visualization”)**.

Additional readings (not mandated, but strongly recommended)

- Nussbaumer Knaflic (2015), *Storytelling with Data* — practitioner reference for narrative-with-data
- Cairo (2012), *The Functional Art* — design and perception foundation
- Wilke (2019), *Fundamentals of Data Visualization* — free online, with a figure-by-figure catalogue
- Schwabish (2021), *Better Data Visualizations* — written for scholar / researcher audiences
- Tufte (2001), *The Visual Display of Quantitative Information* — the classic
- Segel and Heer (2010), “Narrative Visualization: Telling Stories with Data” — an academic foundation

BUILDING A NARRATIVE WITH DATA

WHAT A DECISION-MAKER NEEDS

Three reasons to present your result as a figure rather than a table:

- A board, a regulator, a client, or a senior colleague will not read your code or your CSV — they read the figure instead.
- A table lists the numbers, while a figure shows the pattern in them — the trend, the turning point, or the gap between two series.
- A reader takes in a clear figure in a few seconds, far faster than reading the same result written out in full.

Nussbaumer Knaflic (2015) calls the work of turning raw data into a clear visual conclusion **storytelling with data**. It is a specific craft, learnable, and central to fintech work.

THE CONSULTING SCENARIO

Imagine your first year out of UNSW, working as an analyst at a consulting firm (Big Four, McKinsey, BCG, Bain), an investment bank, the RBA, or Treasury. You download, clean, and analyse economic data, and you produce a result.

The same result now goes to three readers, each of whom needs a different level of detail:

- **The senior partner** will present your work to the client. They need a clear figure and a sentence that states the conclusion.
- **The client's CEO** wants the short version: a small set of figures that move from the question to the answer, without the technical detail.
- **The client's analysts** will check your work. They need every series, its source, its units, and the sample window.

The partner and the CEO do not open your code, tables, or CSV files — they see only the figures and the few sentences around them.

THREE POTENTIAL PITFALLS

Three ways a figure can fail:

- **Wrong question:** the figure shows a level when the question was about a change, or a path when the question was about a relationship. The answer is clear but answers the wrong question.
- **Hidden answer:** the message is in the data, but the design buries it. Too many series crowd the same axis, nothing highlights the series that matters.
- **Untrustworthy figure:** it does not state its source, its sample window, or its units, so the reader cannot check it and will not rely on it.

QUESTION → EVIDENCE → ANSWER

Every figure in this course is built around a three-part arc:

1. Start with the **question** the figure will answer (one question only)
2. Choose the **evidence** — the right series, the right transformation, the right time window
3. Make the **answer** visible in the design — emphasis colour, direct labels, a sentence-form title

The figure is finished when a colleague who has not seen the data can understand the answer within a couple of seconds.

THE STORYTELLING CYCLE

Nussbaumer Knaflic (2015) gives a six-step cycle for building figures:

1. **Understand the context** — who is the audience, and what do they need to do?
2. **Choose an effective visual** — the figure type follows from the question
3. **Eliminate clutter** — chartjunk, unused gridlines, redundant labels (Tufte, 2001)
4. **Focus attention** — one emphasis colour, direct labels, a sentence title
5. **Think like a designer** — alignment, white space, hierarchy (Cairo, 2012)
6. **Tell a story** — sequence figures so each builds on the previous

The first five steps build an individual figure, while the sixth orders the figures into a *sequence*.

CONSTRUCTING A NARRATIVE FROM DATA

Segel and Heer (2010) describe seven common formats for presenting a data-driven narrative:

- Magazine-style article ($\approx$ FT, NYT, The Economist)
- Annotated chart ($\approx$ Bloomberg single-figure explainer)
- Partitioned poster ($\approx$ a one-page macro briefing)
- Flowchart, comic strip, slideshow, video

This week's case study is **magazine-style**: a five-figure sequence, each figure captioned to stand alone, ordered by an underlying question. An FT macro article is built the same way.

A figure that **stands alone** can be read on its own:

- The reader understands it from the figure itself, with no surrounding text
- Everything needed is on the figure — title, units, source, and sample window

THE FT APPROACH

A five-figure FT macro article is built from the same handful of elements:

- **A question** — “Is Australian inflation back to target?”
- **Five figures**, each answering one sub-question of the main question
- **One emphasis colour** repeated across all five
- **Sentence titles** that state the conclusion, not the topic (“Inflation is falling but is not yet back to target”)
- **Source line** on every figure
- **Closing figure** that recaps the story — usually an endpoint comparison

WHAT MAKES A “GOOD” FIGURE

THE FOUR STANDARDS A FIGURE SHOULD MEET

Every figure that appears in a report, a slide, or a chapter should satisfy four conditions:

1. A **caption** that lets the figure stand alone — describes what it shows, the units, and the take-away
2. Axis labels with **units** — for example “Percent”, “Index, 2025 = 100”, or “Basis points”, not a bare “Y”
3. A **source** line — where the data came from, with a date or vintage
4. A **sample window** — exact start and end dates of the data shown

A figure is finished only if a colleague can paste it into a slide, remove your notes, and still understand it. This is what a “self-contained” or “stand-alone” figure means.

ONE QUESTION PER FIGURE

A good figure answers **one** question, and a great figure makes the answer obvious within the first couple of seconds.

- “How does *this* compare to *that*?” → several lines together, or a scatter
- “How has *this* moved over time?” → a single line
- “Where is *this* relative to its own history?” → a series shaded against a reference level
- “How do a few *items* compare across two dates?” → one marked value per item at each date

The figure-type catalogue later in the lecture gives each of these a name. Two questions implies two figures (Schwabish, 2021).

WHAT FT-STYLE MEANS IN THIS COURSE

FT-style is shorthand for the visual conventions used by the *Financial Times* visual journalism team. The FT has spent two decades refining how a figure should look in a serious newspaper, and they publish their conventions openly. You can adopt the six conventions below, all of which you can implement in Matplotlib (a Python plotting package) today:

- Sans-serif font, generous whitespace, no chart-junk (Tufte, 2001)
- A clear legend to identify each series (a direct label works when there is only one line)
- One emphasis colour — for example FT pink (#990F3D) — with muted greys for everything else
- Event shading rather than vertical reference lines for episodes (a recession band, a tightening cycle)
- A title that is a *sentence* (“Inflation has fallen but is not back”), not a topic (“Inflation”)
- Source line and sample window in a small footer, on every figure — no exceptions

The `fintools.figures` helper used in this course implements all six. The `fins-agent` workspace also has this FT-style context built in, so you can produce a compliant figure simply by asking (prompting) the agent in plain language.

THE FT VISUAL VOCABULARY

The FT's visual-journalism team publishes the **Visual Vocabulary** — a free, openly-licensed reference poster + interactive tool that classifies figure types by the *question they answer*, not by the dataset they use.

The same resource is available in two places:

- Interactive web app: <https://ft-interactive.github.io/visual-vocabulary/>
- Source repository (with a downloadable A3 poster, R / Python / D3 code, and licence): <https://github.com/Financial-Times/chart-doctor/blob/main/visual-vocabulary/README.md>

This is the best one-page reference for “which figure type for which question”.

NINE KINDS OF QUESTION A FIGURE CAN ANSWER

The FT Visual Vocabulary organises figures under nine categories. Each category answers a different kind of question:

- **Deviation** — emphasises variation from a fixed reference (e.g. inflation versus the 2–3% RBA target band)
- **Correlation** — shows the relationship between two variables (e.g. commodity prices and the AUD)
- **Ranking** — relative position of items (e.g. countries by GDP per capita)
- **Distribution** — frequency of values within a series (e.g. histogram of monthly returns)
- **Change over time** — trends and trajectories (e.g. the cash rate cycle)
- **Magnitude** — size comparisons at a point in time (e.g. payrolls by industry)
- **Part-to-whole** — composition (e.g. share of GDP by sector)
- **Spatial** — geographic patterns (e.g. unemployment by state)
- **Flow** — movement between states (e.g. Sankey of capital flows)

This week we focus on *Change over time*, *Correlation*, and *Deviation*. The other six categories matter for later work.

HOW TO USE THE VISUAL VOCABULARY

To build a figure, work through these steps:

1. State your question in a sentence (e.g., “How has the cash rate moved through the tightening cycle?”)
2. Match the question to one of the nine FT categories (here, Change over time)
3. Pick a figure type within that category (line, area, slope, fan, calendar, connected scatter)
4. Apply the four standards (caption, units, source, sample window)
5. Export at the right size for the medium (A4 portrait for the chapter, 16:9 for slides)

You can also describe your question to an AI agent and ask which figure type it recommends.

Most “which figure should I use?” confusion comes from skipping step 1: state the question in words first, and the right figure type is usually obvious.

BUILDING A FIGURE STEP BY STEP

A QUICK PLOT CATCHES DATA PROBLEMS

A plot is not only a way to present results. A quick plot of new data is also the fastest way to check that the data is what you expect, and it catches problems that a summary table from `df.describe()` does not always show:

- A misread date column (the dates appear as meaningless values because pandas read the column as text rather than dates)
- An incorrect unit (an interest rate that should read 5% appears as 500 because the basis-point figure was never divided by 100)
- A break or jump that signals a methodology change (e.g. a CPI series spliced from two ABS releases)
- Outliers that should not be there (a single observation at \$10,000 in a price series whose typical value is near \$50)
- A missing-data problem (a flat horizontal stretch in the line where a gap was filled forward with the previous value instead of left blank)

BASIC PLOTTING SKILLS ON SYNTHETIC PRICES

Before building any narrative, you should be able to:

1. Build a tidy DataFrame with columns `date`, `ticker`, and `price`
2. Plot a Matplotlib line, with dates on the x-axis and value on the y-axis
3. Add axis labels, a title, and a sample-window stamp
4. Re-export the same figure in FT style

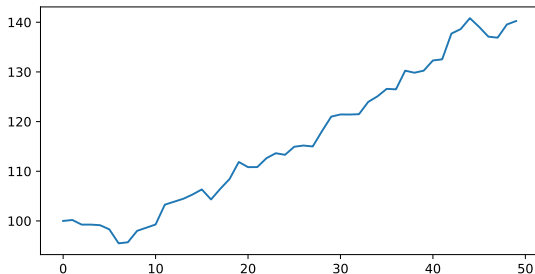
Three Python scripts that come with this course cover these steps on synthetic prices, so you get the pandas and Matplotlib mechanics right before working with macroeconomic data:

- `fins2026/week2/scripts/make_beginner_synthetic_prices.py` — builds the synthetic CSV
- `fins2026/week2/scripts/make_beginner_simple_time_series.py` — plain Matplotlib
- `fins2026/week2/scripts/make_beginner_ft_time_series.py` — FT-style upgrade

WHAT NOT TO DO: A DELIBERATELY BAD PLOT

The script below runs and produces a figure. The figure fails all four standards. We discuss why on the next slide.

```
import pandas as pd, matplotlib.pyplot as plt
df = pd.read_csv(
    "fins2026/week2/results/data/week2_beginner_synthetic_prices.csv")
plt.plot(df["price"])
plt.show()
```



DISSECTING THE BAD PLOT

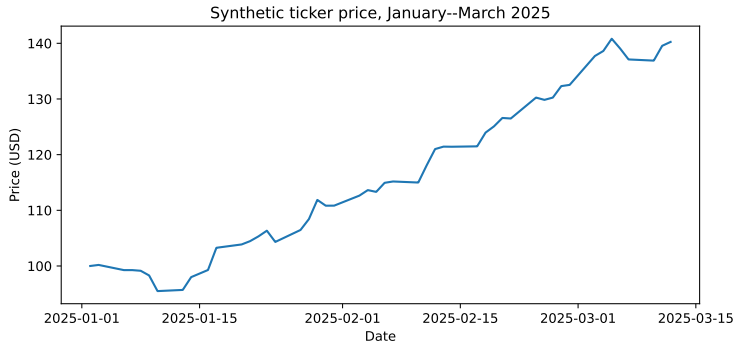
Five problems with the figure on the previous slide, each tied to a missing standard or a data-type error:

- **No date axis.** `plt.plot(df["price"])` uses row numbers (0, 1, 2, ..., 49) on the x-axis. The reader cannot tell which year, month, or day is which.
- **No axis labels.** The y-axis is unlabelled, so the reader cannot tell whether the values are dollars, cents, per cent, or index points.
- **No title.** The figure does not announce what it is or what question it answers.
- **No source, no sample window.** The figure is not defensible in front of a client or a colleague, and not citable in a report.
- **No data-type checking.** `pd.read_csv` returned the date column as a string. Any later `resample` or `merge` on this DataFrame will silently fail.

STEP 1: PLAIN MATPLOTLIB

Run me (from fins-agent): `python fins2026/week2/scripts/make_beginner_simple_time_series.py`

Reads: `fins2026/week2/results/data/week2_beginner_synthetic_prices.csv`



Source: synthetic | Sample: 2025-01-02 to 2025-03-12

Accurate, complete, and readable: with a title, axis labels, and a sample-window stamp, this is the baseline for any figure you build in this course.

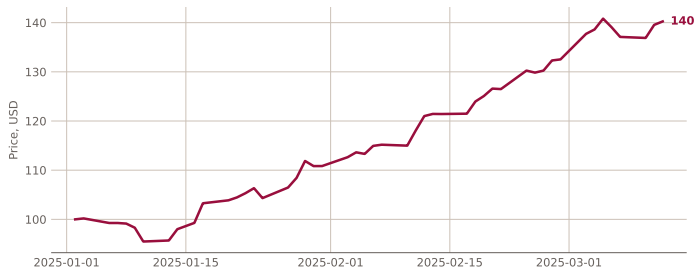
STEP 2: FT-STYLE UPGRADE

Run me (from fins-agent): `python fins2026/week2/scripts/make_beginner_ft_time_series.py`

Reads the same synthetic CSV and **writes** a PDF and a PNG

The synthetic ticker rose 40% over the sample window

Synthetic single-ticker price, business days



Source: synthetic data Sample: 02 Jan 2025 to 12 Mar 2025

The data and DataFrame are identical to Step 1 — only the presentation has changed: a sentence title that states the finding, FT pink as the emphasis colour, sans-serif type, a footer with the source and sample window, and a direct value label at the right end of the line.

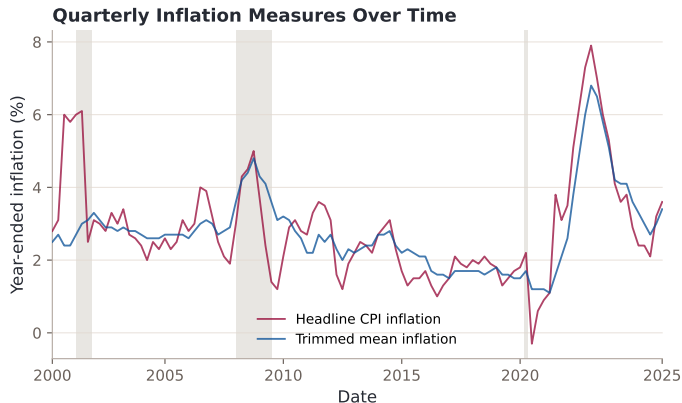
COMPARING THE THREE VERSIONS

Three figures of the same data, in order:

- **Bad plot** — `plt.plot(df["price"])` alone, with row indices on the x-axis and nothing else — fails every standard
- **Plain Matplotlib (Step 1)** — adds dates on x, axis labels with units, a topic title, and a source line — the minimum acceptable figure
- **FT-style upgrade (Step 2)** — replaces the topic title with a sentence finding, drops chartjunk, picks one emphasis colour, and labels the endpoint directly — the minimum publication-grade figure

Across the three figures the data does not change — only the presentation does, from unreadable to publication-grade.

FT CONVENTIONS APPLIED TO MACROECONOMIC DATA



The six conventions in the figure.

- Sentence title states the finding
- FT pink as the emphasis colour
- Clear legend/line labels
- RBA 2–3% target band shaded
- Sans-serif type, no chart-junk
- Source line and sample window in the footer

The data: RBA monthly headline and trimmed-mean CPI through December 2025. Every figure later in this lecture follows this pattern.

THE AUSTRALIAN MACRO TOOLKIT

AUSTRALIAN MACRO: THIS WEEK'S DATASET

This course is taught at UNSW Business School in Sydney. The Australian economy is the most relevant to you here, and the economy a graduate employer will expect you to know. It is the dataset we use this week.

Three industry scenarios this week prepares you for directly:

- **Asset management.** Your head portfolio manager asks you for a one-page read on the Australian macro environment ahead of the next investment-committee meeting. The page must fit five figures, with three sentences on each.
- **Macroeconomic forecasting.** You join a forecasting consultancy that sells AUD/AU-rates outlooks to asset managers. Before you forecast anything, you have to describe where the economy is right now: inflation regime, labour market, cash-rate cycle, external sector.
- **Macro hedge fund.** The fund decides how to allocate risk by country. The country case begins with macro fundamentals — growth, inflation, terms of trade, policy stance — presented as a coherent set of figures rather than a table of raw numbers.

In each scenario the deliverable is the same: a small, defensible set of figures that lets a senior decision-maker see the state of the economy quickly.

THREE REASONS WE USE AUSTRALIAN DATA

Three reasons the Australian macroeconomy works well for this lecture:

- The RBA publishes its macroeconomic statistics as clean Excel tables, free, with stable series identifiers — an ideal Stage 1 data source
- The Australian release calendar covers the full range of mixed-frequency and release-lag cases this chapter teaches: daily, monthly, quarterly, and point-in-time series
- The sample spans the 2022–2024 tightening cycle and the 2025 disinflation, so the figures show large, recent moves rather than a flat stretch of data

The next slide defines the twelve series used this week.

AUSTRALIAN MACRO VARIABLES: WHAT EACH SERIES CAPTURES

Every series below is published by the RBA at <https://www.rba.gov.au/statistics/tables/>. The Table column is the RBA's identifier (e.g. table F1.1 = Interest Rates and Yields – Money Market). Cadence is the native frequency before any resampling.

Series	Table	Cadence	What it captures
Cash rate target	F1.1	daily	The RBA's main policy rate — its target for the overnight rate banks lend to each other at
10Y govt bond yield	F2.1	daily	The cost of long-run government borrowing, reflecting growth and inflation expectations
Trade-weighted index	F11	monthly	The AUD against a trade-weighted basket of partner currencies
Headline CPI	G1	quarterly	Cost of a representative consumer basket, volatile components included
Trimmed-mean CPI	G1	quarterly	The RBA's preferred inflation gauge, trimming the most volatile 15% of components each side
Real GDP (level)	H1	quarterly	The size of the Australian economy in chain-volume terms
Real GDP growth (y/y)	H1	quarterly	Year-ended log change of real GDP — the activity headline
Wage Price Index	H4	quarterly	A measure of wage growth that controls for compositional shifts
Unemployment rate	H5	monthly	Share of the labour force that is unemployed — the headline labour gauge
Participation rate	H5	monthly	Share of the working-age population in the labour force
Job vacancies	H5	quarterly*	Unfilled jobs counted at a single date — a snapshot, not a total accumulated over the quarter
Commodity prices (A\$)	I2	monthly	Price of Australia's resources export bundle, a key driver of the AUD

FOUR FUNCTIONAL GROUPINGS OF THE TWELVE SERIES

The twelve series fall into four groupings:

- **Monetary stance** (cash rate, 10Y yield): what the RBA is doing now, and what the market expects it to do next.
- **Inflation regime** (headline and trimmed-mean CPI): what households actually pay, and the underlying trend the RBA targets.
- **Real economy and labour** (GDP, unemployment, participation, vacancies, WPI): the evidence the RBA uses to judge whether tight policy (a high cash rate meant to slow demand and lower inflation) is feeding through.
- **External sector** (trade-weighted index, commodity prices): for a small open commodity exporter these two move together, with consequences for inflation and the cash-rate path.

READING DATA FROM THE WEB

THE TWO PUBLIC DATA SOURCES USED THIS WEEK

This week's data is drawn from two public sources. The RBA is the primary source because the Australian macroeconomy is the workshop dataset. FRED is the secondary source, used for the US comparison exercises.

- **Reserve Bank of Australia (RBA) statistical tables.** Excel files, grouped by topic (F: financial markets, G: prices, H: economic activity, I: external sector). *Primary source.*
- **Federal Reserve Economic Data (FRED).** A no-key CSV endpoint maintained by the Federal Reserve Bank of St. Louis. *Secondary source.*

Reading from the RBA and from FRED follows the same workflow:

- URL → pandas DataFrame → tidy long table → save to disk.
- The lecture, the exercises, and the case study then read the saved file, so everything runs offline from the saved copy.

READING DATA FROM A URL

Up to now, most `pd.read_csv(...)` or `pd.read_excel(...)` calls you have written have read a file from *your hard drive*:

```
df = pd.read_csv("data/prices.csv")
```

This week, we pass a **URL** instead of a local path:

```
df = pd.read_csv("https://fred.stlouisfed.org/.../fredgraph.csv?...")
```

What pandas does internally:

- Opens an HTTP connection to the URL.
- Downloads the file into a temporary buffer in memory.
- Parses it as a CSV (or Excel) file and returns a DataFrame.
- Discards the buffer; the file is not written to your disk unless you save it yourself.

A `pd.read_csv` or `pd.read_excel` call works against a local path or a URL without any code change — if you can read a CSV from disk, you can read it from the web.

WHERE THE RBA PUBLISHES ITS DATA

The RBA publishes its statistical tables at:

<https://www.rba.gov.au/statistics/tables/>

Each table has a stable URL. The cash rate (table F1.1) is at:

<https://www.rba.gov.au/statistics/tables/xls/f01hist.xlsx>

Notice the URL pattern: the file is an `.xlsx`, the path encodes the table family (`f01`), and `hist` signals the historical (long-run) version. Most other RBA tables follow the same convention.

The RBA URL is the input to your Stage 1 pipeline. Open the table in your browser first, see what the spreadsheet looks like, and only then write the `pd.read_excel` call.

READING AN RBA TABLE

Run me (from fins-agent): `python fins2026/week2/scripts/pull_australia_macro_data.py`

Downloads every RBA series we need and saves a long table with correct column types to data/

```
import pandas as pd
URL = "https://www.rba.gov.au/statistics/tables/xls/f01hist.xlsx"
# RBA tables have a ~10-row header block, then the data.
# header=10 tells pandas to start parsing at row 11.
raw = pd.read_excel(URL, header=10)
# Clean: rename to snake_case, parse dates, drop metadata rows
cash_rate = (raw
    .rename(columns={"Series ID": "date", "FIRMMCRT": "cash_rate"})
    .assign(date=lambda d: pd.to_datetime(d["date"], errors="coerce"))
    .dropna(subset=["date"])          # drops the metadata rows where date is NaT
    .set_index("date").sort_index()
)
print(cash_rate[["cash_rate"]].tail().to_string())
```

Notes: FIRMMCRT is the RBA's series code for the cash rate target (we look these up from the RBA documentation). `errors="coerce"` turns un-parseable date strings into NaT, which the next `dropna` removes. This approach works for every RBA F / G / H / I table. Only the header offset and the column code change.

READING THE FRED MARKET PANEL

Run me (from fins-agent): `python fins2026/week2/scripts/pull_fred_market_data.py`
Downloads the ten US series and saves to `data/fred_market_macro.csv`

```
from fintools.apps import read_fred_graph_csv

SERIES = ["DGS10", "DGS2", "DTB3", "T10Y2Y", "VIXCLS",
          "UNRATE", "INDPRO", "PAYEMS", "FEDFUNDS", "SP500"]

fred = read_fred_graph_csv(SERIES).set_index("release_date").sort_index()
```

Result: a wide-form DataFrame with ten columns (one per series), indexed by calendar date. Daily series (rates, VIX, S&P 500) carry a value each trading day. The monthly series (UNRATE, INDPRO, PAYEMS) sit at NaN between releases (McKinney, 2022).

STAGE 1 OF THE DATA FACTORY FLOOR

THE EIGHT STEPS OF STAGE 1

Week 1 introduced Station 1 of the Data Factory Floor: the stage that turns a raw source file (e.g. an RBA Excel sheet or a FRED CSV) into a clean, audited input that downstream code can rely on. It does three things: it converts the date columns to a `datetime` type (pandas reads them as plain text by default), it reshapes the rows into a tidy long table with a known schema, and it runs a set of automatic checks that confirm the data matches what we expected.

1. PULL — download the source file from the URL
2. PARSE — strip the header, footer, and metadata blocks
3. ASSIGN COLUMN TYPES — give every column an explicit data type
4. DATE-MAP — record reference, release, and observable dates
5. ERROR CHECK — run dtype, range, and duplicate assertions
6. LAG CHECK — record the per-series publication lag
7. FREEZE — set the common reference and observable endpoints
8. SAVE — write the long table (correct column types) to Parquet

An error fixed in Stage 1 is a quick edit at the data step. The same error found after the figures are built forces a rebuild of the whole figure pipeline.

STEPS 1–3: PULL THE FILE, PARSE THE ROWS, ASSIGN COLUMN TYPES

- **Pull.** A network call: `pd.read_excel(URL)` or `pd.read_csv(URL)` returns a `DataFrame`.
- **Parse.** Drop the metadata rows, rename the columns to clean snake_case identifiers, drop empty rows. RBA tables have a 10–12 row header block. FRED CSVs are clean and need no header stripping.
- **Assign column types.** Only the date column needs manual work — pandas reads it as text, so convert it to `datetime64[ns]` with `pd.to_datetime(..., errors="coerce")`. pandas already reads the numeric columns as `float64` on its own, so a quick `df.dtypes` check is all they need.

After these three steps the raw spreadsheet has become a clean wide-form `DataFrame`. It is not yet a tidy long table, which Step 4 produces.

STEP 4: DATE-MAP

Every observation in this week's Stage 1 long table has three timestamps:

- `reference_date` — when the economic period *happened* (e.g. 2025-12-31 for Q4 GDP)
- `release_date` — when the source first *published* the value (e.g. 2026-03-04)
- `observable_month_end` — the *classroom* month-end when we treat the value as available (here: 2026-03-31)

Two figures from the same data answer different questions:

- “What happened in Australia in 2025?” → plot by `reference_date`
- “What did a researcher know on each month-end?” → plot by `observable_month_end`

STEP 5: ERROR CHECKS

Run me (from fins-agent): `python fins2026/week2/scripts/make_australia_stage1_walkthrough.py`
After loading the data, run these four assertions before anything else uses it:

```
# 1. dtype check: the date column must be a datetime, not a string
assert long.dtypes["reference_date"] == "datetime64[ns]"

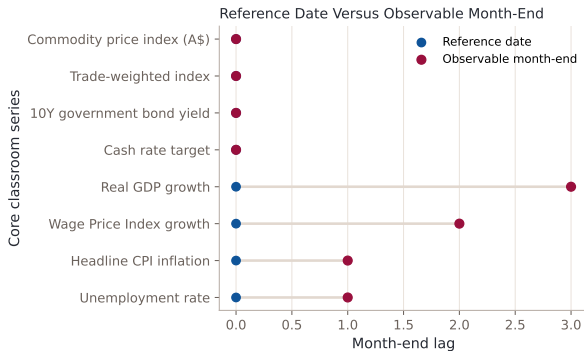
# 2. completeness check: at most 5% missing.
#    0.95 is our chosen threshold: if more than 5% of values came back
#    NaN, the read failed somewhere (wrong header offset, a 404, or
#    a column rename gone wrong) and the long table cannot be trusted.
assert long["value"].notna().mean() > 0.95

# 3. ordering check: a value cannot be released before it is observed
assert (long["reference_date"] <= long["release_date"]).all()

# 4. uniqueness check: one value per (series_id, reference_date)
assert not long.duplicated(subset=["series_id",
                                   "reference_date"]).any()
```

The four assertions catch the common Stage 1 errors: a date column left as text, a file that loaded with mostly missing values, a value released before the period it measures, and duplicate rows for the same series and date.

STEP 6: LAG CHECKS



Each Australian series releases on its own calendar: the cash rate, 10Y yield, TWI, and commodity index publish live (0-month lag), unemployment and monthly CPI lag 1 month, quarterly CPI and WPI lag 1–2 months, and GDP lags 3 months. Stage 1 records the lag in the long table so figures can choose reference-date or observable-date views.

LOOK-AHEAD BIAS: WHAT IT IS

Look-ahead bias is when an analysis uses a piece of information that was not yet available at the moment the decision was supposedly being made.

For a concrete example, suppose today is **31 January 2026**, and you are studying whether GDP growth predicts equity returns.

- The GDP release for Q4 2025 (reference date 31 December 2025) will not be published until **4 March 2026** — a 3-month lag.
- If you use that GDP number to “predict” January 2026 returns, your model is cheating: it is using a number that no analyst, fund, or trader could possibly have known on 31 January.
- The model will appear excellent in backtest and underperform out of sample.

Look-ahead bias is a common reason a backtest looks strong but live trading underperforms. Recording each series' release lag and merging on the observable date prevents it.

USING THE RELEASE-LAG FIELDS DOWNSTREAM

Look-ahead bias is a Stage 1 task, handled in the data rather than in the model. The same three-field convention then governs the work downstream:

- **In this week's figures.** Every figure in the Australia narrative respects the release-lag structure. “What did Australia look like on 31 December 2025?” is plotted by `reference_date`. “What could a researcher have known by 31 March 2026?” is plotted by `observable_month_end`.
- **In any out-of-sample evaluation.** Any merged panel that claims to be real-time must use `observable_month_end` as its merge key. A model that trains on revised data plus future releases is not forecasting.
- **In any backtest, event study, or portfolio simulation.** The same three-field convention ties every result to what was knowable at the time.

If your panel's merge key is `reference_date` when it should be `observable_month_end`, every downstream figure and model is wrong. Carrying all three dates and merging on `observable_month_end` prevents that error.

STEP 7–8: FREEZE AND SAVE

The pipeline fixes two endpoint dates so the figures are the same no matter when you run it:

- **Common reference endpoint:** 2025-12-31
- **Common observable endpoint:** 2026-03-31

Stage 1 saves the long table to a Parquet file that keeps each column's type, so downstream code reads it without re-parsing the spreadsheet. The frozen long-table fixture is published on Canvas.

Stage 1 produces a tidy long Parquet file with its schema, sample window, and timing fields locked down, which every downstream figure and analysis loads from.

THE FORWARD-FILL TRAP

A common mistake when merging quarterly GDP onto a monthly panel:

```
# DO NOT do this without thinking  
panel["GDP"] = panel["GDP"].ffill()
```

- `ffill()` copies the last observed quarterly value forward into the months that have no GDP reading.
- The monthly series then repeats each quarter's value for three months, so a line chart looks flat for two months and steps up at the next quarter.
- That step pattern comes from the fill rule, not from the economy, and the chart gives the reader no way to tell the two apart.

To show a quarterly series on a monthly chart, plot it as a step line that holds each value until the next quarter, or as points at the quarter-end dates, so the chart does not imply data you do not have.

RESAMPLING AND MERGING

THE NEXT THREE SECTIONS USE THE US PANEL

The Australian Stage 1 table is built. The next three sections — resampling and merging, the transformation menu, and the figure-type catalogue — demonstrate the Station 2 toolkit on the **US/FRED panel** rather than the Australian data.

Two reasons for the switch:

- The FRED panel has a clean mix of daily series (rates, the S&P 500) and monthly series (unemployment, industrial production) — exactly what the resampling and merging steps need to show.
- Several figure types have a natural US example — the Treasury yield-curve inversion, the federal-funds-rate-versus-unemployment scatter — that the RBA tables do not provide.

The Australian data returns for the full five-figure narrative once the toolkit is complete.

DAILY → MONTHLY WITH RESAMPLE

See (from fins-agent): `fins2026/week2/code/market_panel.py` (resample + merge helper used by the figure scripts)

```
# "ME" is the month-end frequency alias in modern pandas

# Last observation of each month -- right for stocks, rates
monthly_levels = daily.resample("ME").last()

# Mean of each month -- right for flow series, signal averages
monthly_means  = daily.resample("ME").mean()
```

The choice depends on the series:

- Equity index, bond yield, exchange rate → `.last()` (point-in-time)
- Trading volume, daily turnover → `.sum()`
- Volatility, sentiment → `.mean()` (average state)

MERGING ON THE MONTH-END KEY

Run me: same script (07_resample_merge.py), which produces the merged **US** teaching panel (2015–2025)

```
panel = (  
    monthly_market                # daily resampled to ME  
    .join(monthly_macro, how="outer") # macro on the same key  
    .sort_index()  
)  
panel = panel.loc["2015-01":"2025-12"] # frozen US window
```

Two design choices:

- `how="outer"` keeps every month even if either side is missing
- Slicing with `.loc[start:end]` freezes the sample window before any figure is built

The merge happens once, at the top of the script. Every figure downstream draws from the same panel.

CHOOSING A TRANSFORMATION

WHEN LEVELS WORK AND WHEN THEY MISLEAD

Whether to plot a raw *level* depends on the question and the series:

- Levels are right when the level itself is the question — “what is the cash rate today?”
- Levels are wrong when the units are not comparable — GDP in millions next to CPI in per cent
- Levels are wrong when the change is too small to see in the raw level — a series that drifts slowly over time
- Levels mislead on compounding series — on a linear axis the same percentage gain looks larger at a higher price, so a log scale or a returns transform is clearer

Match the transformation to the question — a level for “where is it now”, a change for “how much did it move”, a growth rate for “how fast is it growing”.

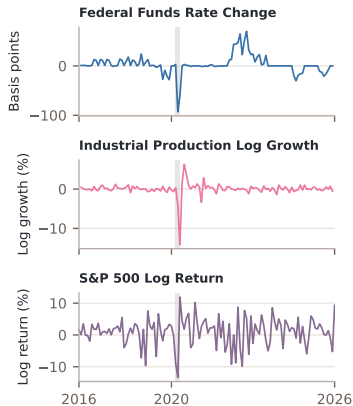
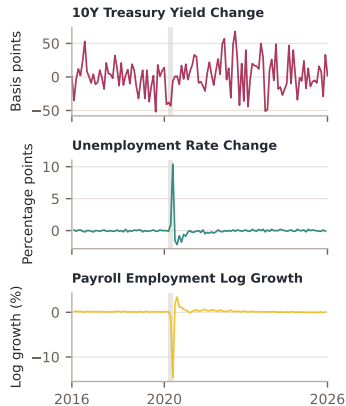
THE DECISION TABLE

Question	Transformation	Series example
Where is the level today?	<i>Level</i>	Cash rate
How much did it move?	<i>First difference</i>	10Y yield, unemployment
How fast is activity growing?	<i>Log growth (year-ended)</i>	Real GDP
How did equities perform?	<i>Simple or log return</i>	S&P 500
How do mixed-unit series compare?	<i>Indexed (start = 100)</i>	GDP, CPI, payrolls
What was wealth growth?	<i>Cumulative growth-of-\$1, log y</i>	S&P 500

Pick the transformation from the question rather than from the dataset, and only then build the figure.

FIVE TRANSFORMATIONS OF THE MERGED PANEL

Month-End Transformations By Series



The same merged month-end U.S. panel under five transformations:

- Levels: where the series sits today
- First difference of a rate: speed of rate moves
- First difference of unemployment: labour moves
- Log growth: activity trend
- Simple return: equity performance

THE TRANSFORMATION CODE PATTERN

See (from fins-agent): `fins2026/week2/code/market_window.py` (transformations helper used by the figure scripts)

```
import numpy as np

# First difference of a rate. DGS10 is in per cent, so .diff() gives the
# change in percentage points, and *100 reports that change in basis points.
panel["DGS10_BP"] = panel["DGS10"].diff() * 100

# First difference of the unemployment rate, in percentage points.
panel["UNRATE_PP"] = panel["UNRATE"].diff()

# Year-ended log growth:  $\log(\text{level}_t / \text{level}_{t-12}) * 100$ , in per cent
panel["INDPRO_LG"] = np.log(panel["INDPRO"]).diff(12) * 100

# Simple monthly return for equities  $(\text{price}_t / \text{price}_{t-1} - 1)$ 
panel["SP500_RET"] = panel["SP500"].pct_change()
```

Each transformation is three lines, and every figure this week starts from these derived columns.

LOG Y FOR COMPOUNDING

When plotting the cumulative growth of \$1, use a **log scale** on the y-axis, not a linear scale.

- **Why.** Returns compound multiplicatively. On a log axis, equal percentage changes take equal vertical space, so a 50% drawdown is the same height whether wealth is at \$10 or \$1,000.
- **Reading the slope.** The slope is the growth rate: a constant slope is constant compounding, a flatter line means returns are slowing, a steeper line means they are accelerating.
- **Reading the level.** Tick labels stay in dollars (\$1, \$2, \$5, \$10). The rightmost point is the final value: \$3.20 means \$1 grew to \$3.20, a total return of 220%.
- **Code rule.** Plot growth of \$1, `(1 + r).cumprod()`, not cumulative return, `(1 + r).cumprod() - 1`. Cumulative return goes negative in drawdowns, and `plt.semilogy` drops the negative values, leaving gaps.

A CATALOGUE OF FIGURE TYPES

A FIGURE TYPE FOR EACH KIND OF QUESTION

Figure type	Use it when	Example
Single line	one series, one question	Cash rate over time
Multi-line	two to four series, same units	Treasury yield curve
Indexed	multiple series, mixed units	GDP vs CPI vs payrolls
Deviation area	threshold or zero crossings	10Y–2Y inversion
Small multiples	many series, same x-axis	Macro pack, one panel each
Scatter w/ fit	relationship, not path	FedFunds vs UNRATE

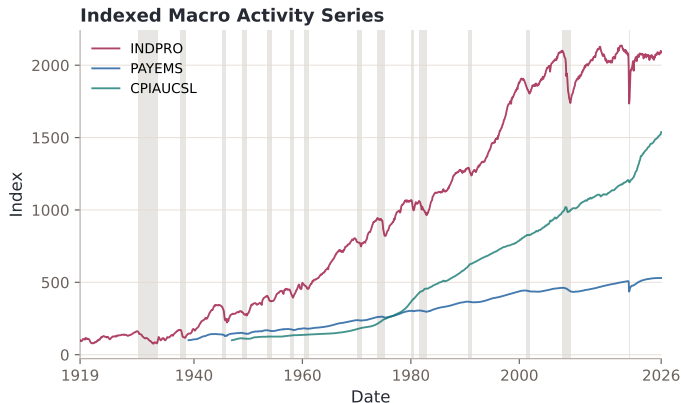
Decision rules adapted from Wilke (2019) and Schwabish (2021). Each row also maps to a `fintools.figures` helper.

WHAT THE NEXT FIVE FRAMES SHOW

The next five frames show one worked example of each figure type from the table on the previous slide. Each example uses publicly available data you can rebuild, and each is captioned with what it shows, the source, and the sample window.

- **Indexed series** — US macroeconomic activity from FRED (industrial production, payrolls, etc.), each indexed to 100 at a base date
- **Deviation area** — the US 10-year minus 2-year Treasury yield spread, FRED daily data
- **Small multiples** — ten US industry portfolios from the Ken French data library, monthly returns
- **Scatter with a fit line** — US monthly federal funds rate against the unemployment rate, FRED data
- **Dumbbell and slope figure** — World Bank GDP per capita comparisons across countries, two endpoint dates

INDEXED SERIES



When to use. Three or more series with mixed units that share a sensible base date.

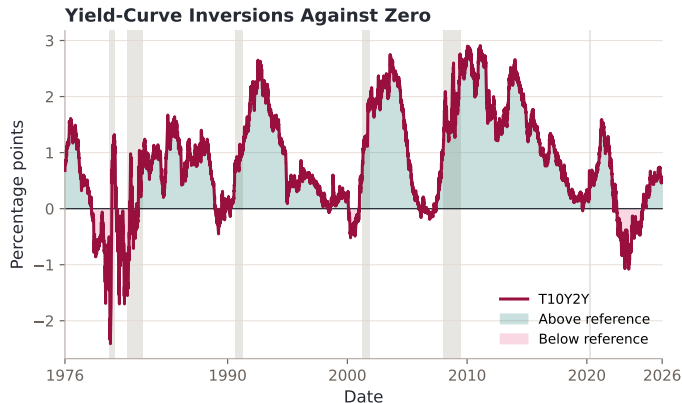
What it shows. Each series rebased to 100 at the start of the sample.

Read this figure by.

- Comparing line slopes, not levels
- Identifying the fastest-growing series since the base date
- Spotting series that fell below 100 and have not recovered

Source. FRED monthly macro panel.

DEVIATION AREA



When to use. A single series whose threshold or zero-crossing is the question.

What it shows. 10-year minus 2-year Treasury spread, shaded above and below zero.

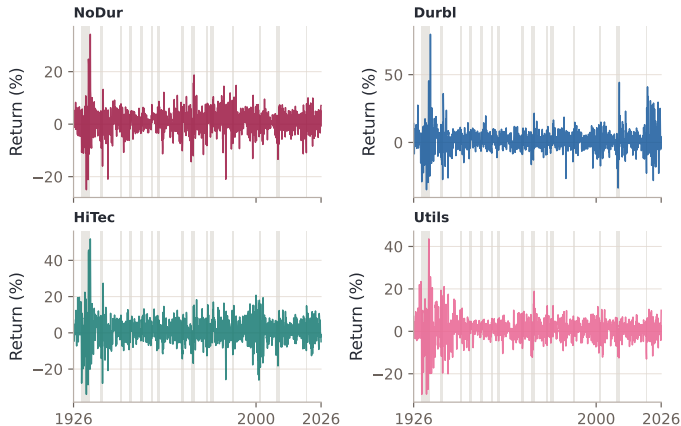
Read this figure by.

- Counting the inversion (red) episodes
- Comparing the duration of each inversion
- Reading the depth of the deepest negative spread

Source. FRED daily T10Y2Y.

SMALL MULTIPLES

Selected Industry Return Small Multiples



When to use. Many series that share an x-axis but not a y-scale.

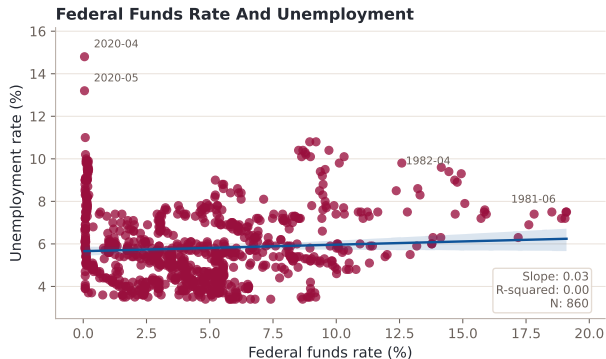
What it shows. Ten US industry portfolios since 1990, one panel each.

Read this figure by.

- Scanning panel-by-panel to see how each industry's returns evolved over time
- Spotting common drawdown episodes that line up vertically (2000, 2008, 2020)
- Reading magnitudes within a panel, not across panels

Source. Ken French data library, ten-industry portfolio returns.

SCATTER WITH A LINE OF BEST FIT



When to use. A relationship between two variables, rather than a path through time.

What it shows. Federal funds rate against unemployment, one month per point.

Read this figure by.

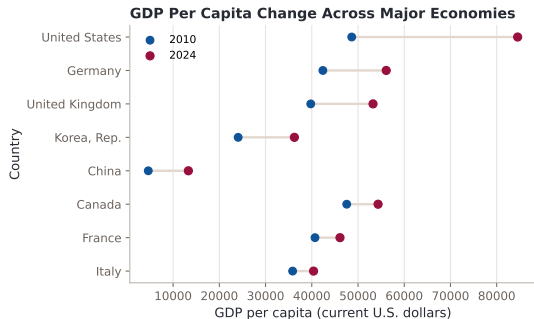
- Reading the overall tilt: the fitted line is nearly flat (slope 0.03, $R^2 = 0.00$), so the two rates show little linear relationship.
- Spotting the labelled outliers: 2020 (near-zero rate, double-digit unemployment) and 1981–82 (high rate, high unemployment) sit far from the main mass of points.
- Treating the slope as a **visual aid only** — it guides the eye toward the relationship, and we draw no statistical claim from it.

Source. FRED monthly FEDFUNDS and UNRATE.

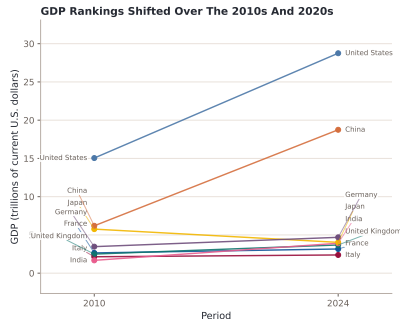
CROSS-SECTIONAL FIGURES: DUMBBELL AND SLOPE

What they show. Time-series figures dominate this week, but two cross-sectional types are worth knowing for endpoint comparisons. Both figures show GDP per capita for a panel of countries between two snapshot dates — 2000 and the most recent year — using World Bank data.

Dumbbell — one bar per country with a line connecting the two endpoints; shows magnitude and direction of change.



Slope figure — one line per country, two vertical axes (start year on the left, end year on the right). Each line's slope shows the direction and size of the change, and crossings flag a rank switch.



THE AUSTRALIAN MACRO NARRATIVE

THE QUESTION

“Tell the story of the Australian economy over time, up to December 2025, in five figures.”

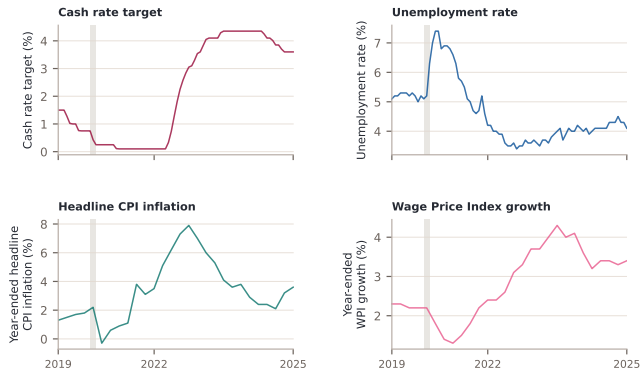
Each figure covers one part of the macroeconomy:

1. **F1** — the cash rate and its macro backdrop (rate, jobs, inflation, wages)
2. **F2** — the inflation regime (headline vs trimmed mean)
3. **F3** — the labour market (four panels)
4. **F4** — the currency and commodity prices (TWI vs commodity scatter)
5. **F5** — December 2000 vs December 2025 (endpoint comparison)

Each figure stands alone with its own caption, and the five together form the six-paragraph written narrative you produce this week, which uses no model.

F1 — THE CASH-RATE CYCLE

Pandemic To Tightening In Australia



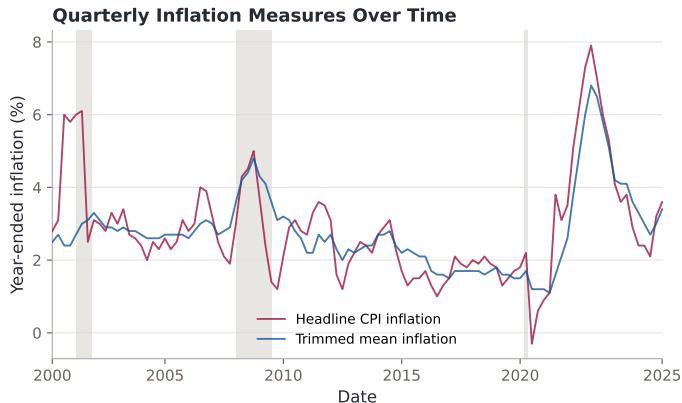
Question. How did the cash rate move through the pandemic and the 2022–2024 tightening, and what was the economy doing around it?

Four panels.

- **Cash rate** (top left): near zero in 2020–21, then up to a $\sim 4.3\%$ peak and a post-2024 plateau
- **Unemployment** (top right): a 2020 spike above 7%, down to a $\sim 3.5\%$ trough, then drifting up
- **Headline CPI** (bottom left): a climb to $\sim 7.8\%$ in 2022, disinflation since
- **Wage Price Index** (bottom right): from $\sim 2\%$ to above 4%, easing toward 3.3%

Source. RBA Tables F1.1, H5, G1, H4; 2019–2025.

F2 — THE INFLATION REGIME



Question. Has Australian inflation returned to target?

What to look at.

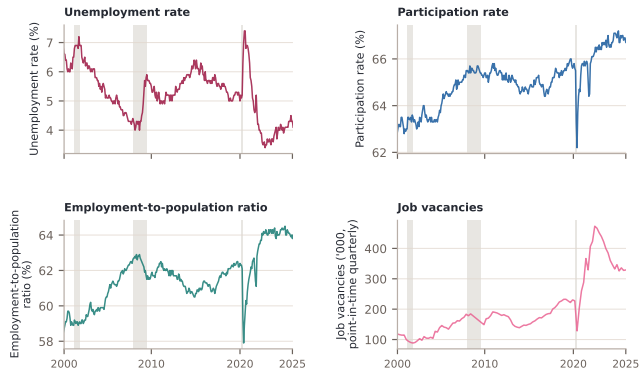
- Headline CPI sits near the upper edge of the 2–3% target band
- Trimmed-mean CPI is marginally closer to target
- Both peaked in 2022 and have been falling since

Source. RBA Table G1, 2000-Q1 to 2025-Q4, quarterly.

Why the trimmed mean. It drops the most volatile 15% of components and is the gauge the RBA itself targets.

F3 — THE LABOUR MARKET

Labour Tightness Needs Mixed Frequencies



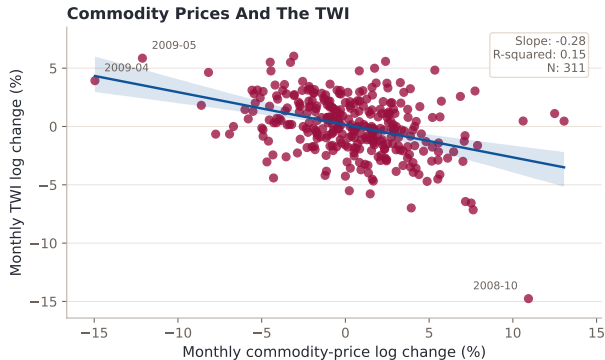
Question. What state is the Australian labour market in by late 2025?

What to look at.

- Four panels on a shared x-axis: unemployment, participation, employment-to-population, and vacancies
- Job vacancies are quarterly; the other three series are monthly
- Unemployment is below its 2000-decade range
- Vacancies are off the 2022 peak but still elevated

Source. RBA Table H5, 2000-01 to 2025-12.

F4 — THE AUSTRALIAN DOLLAR AND COMMODITY PRICES



Question. Does the AUD track commodity prices, and if so, how closely?

What to look at.

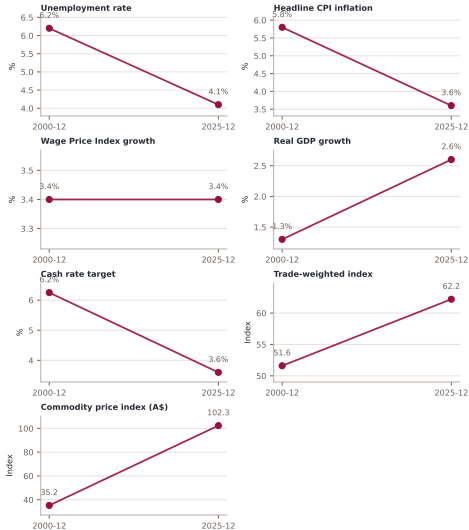
- Each point is one month, 2000–2025 ($N = 311$): the x-axis is the commodity-price log change, the y-axis the TWI log change
- The fitted line slopes slightly down (slope -0.28 , $R^2 = 0.15$), a visual aid only

What it means. The “commodity currency” story predicts the AUD should rise when commodity prices rise — a positive slope. At the monthly frequency it does not appear: the slope is weakly negative and the R^2 is low, so month-to-month commodity moves explain little of the AUD’s moves. The link shows up over longer horizons, not month to month.

Source. RBA Tables F11 and I2, monthly log-changes.

F5 — DECEMBER 2000 VS DECEMBER 2025

December 2000 Versus December 2025



Question. How does Australia in late 2025 compare to December 2000?

What to look at.

- Seven series, each shown at its December 2000 and December 2025 values, with direct labels on both points
- Lower in 2025: unemployment (6.2% → 4.1%), headline CPI (5.8% → 3.6%), the cash rate (6.2% → 3.6%)
- Higher in 2025: real GDP growth (1.3% → 2.6%), the TWI, and commodity prices (the A\$ index roughly tripled, 35 → 102)
- Wage growth unchanged (3.4% → 3.4%)

Source. RBA, 2000-12-31 and 2025-12-31.

INTERPRETING THE FIVE FIGURES WITH A NARRATIVE

The five figures combine into the following narrative:

1. Rates rose fast, then plateaued (F1)
2. Inflation came down but not all the way (F2)
3. The labour market loosened a little, but unemployment stayed below its 2000-decade range (F3)
4. The currency and commodity prices show only a modest monthly relationship (F4)
5. Australia in 2025 looks structurally different from Australia in 2000 (F5)

Your case-study deliverable is five well-built figures and the written narrative that ties them together, with no modelling or forecasting layer required this week.

EXAMPLE PROMPT TO GENERATE THE AUSTRALIAN MACRO REPORT

Most of the figure code is already written this week, so you can hand the whole task to your AI agent. A prompt like this is enough:

```
Save everything under week2/scratch. Make a new folder called aussie_macro. Produce all the figures needed to evaluate the Australian macro environment, then write a neat .docx report that evaluates and discusses the current state of the Australian macroeconomy. Use FT-style figures throughout. Much of the code is already written in week2.
```

Then check every figure and paragraph it produces against the four standards, and fix what is wrong.

PRACTICE: A 10-YEAR VIEW OF THE U.S.
MACROECONOMY

THE PRACTICE ASSIGNMENT

The Australian narrative we just built is the in-class example. **Your assignment** is to do the same thing for the United States.

The task: run a 10-year US macroeconomic dataset through Stages 1 and 2 of the Data Factory Floor, and produce a five-figure narrative of the U.S. economy from 2015 to 2025.

The data comes from the Federal Reserve Economic Data (FRED) service maintained by the Federal Reserve Bank of St. Louis. You read it directly from the URL introduced in Part F: ten series covering Treasury rates, the federal funds rate, unemployment, industrial production, payrolls, the S&P 500, and the VIX.

The deliverable: a Word document (.docx) containing the five captioned figures and a six-paragraph narrative tying them into a story.

STAGE 1 QUESTIONS: GET THE DATA RIGHT

Before you build any figures, complete each of these Stage 1 steps and answer the question in writing:

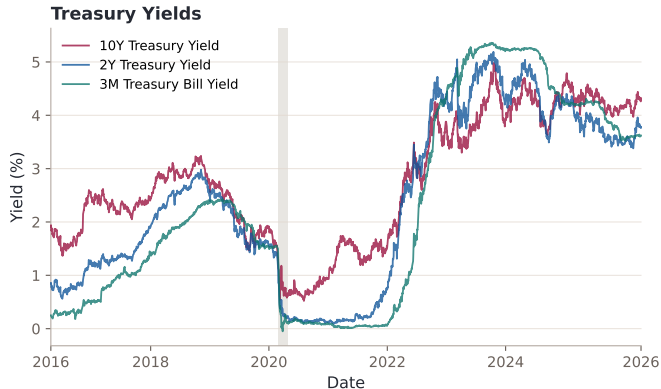
1. **Pull.** Fetch the 10-series FRED panel from <https://fred.stlouisfed.org/graph/fredgraph.csv?id=DGS10,DGS2,DTB3,T10Y2Y,VIXCLS,UNRATE,INDPRO,PAYEMS,FEDFUNDS,SP500>. *Q: how many rows did you receive, and what is the date range?*
2. **Type.** Convert the date column to `datetime64`, and confirm the numeric columns came in as `float64`. *Q: were any values dropped during type conversion? If so, which series and why?*
3. **Sanity-check.** Run the four assertions from Part G (`dtype`, `completeness > 95%`, `ordering`, `uniqueness`). *Q: do they all pass?*
4. **Lag-check.** Look up the publication lag for each FRED series. *Q: which two series have the longest publication lags, and what are they?*
5. **Freeze.** Clip the panel to a 10-year window ending December 2025. *Q: what is the first and last date your figures will show?*

STAGE 2 QUESTIONS: BUILD THE FIGURES

Once Stage 1 is done, build five figures. Each should pick the transformation appropriate to its question:

1. **Treasury yield panel** (multi-line): plot DGS10, DGS2, and DTB3 in levels. *Q: how does the term structure today compare to 2015?*
2. **Yield-curve inversion** (deviation area): plot T10Y2Y shaded above and below zero. *Q: how many distinct inversion episodes are there in the sample, and how deep was the deepest?*
3. **S&P 500 growth of \$1** (line on log y): plot $(1 + \text{SP500_RET}).\text{cumprod}()$ on a log y axis. *Q: what was the worst peak-to-trough drawdown in the sample?*
4. **Federal funds rate vs unemployment** (scatter with fit): plot the two as a scatter. *Q: does the relationship look stable across sub-periods, or does it shift?*
5. **S&P 500 return vs change in VIX** (scatter with fit): monthly. *Q: what does the slope tell you about the equity-volatility relationship?*

WORKED EXAMPLE: TREASURY RATES



Question. How has the U.S. term structure evolved over the past decade?

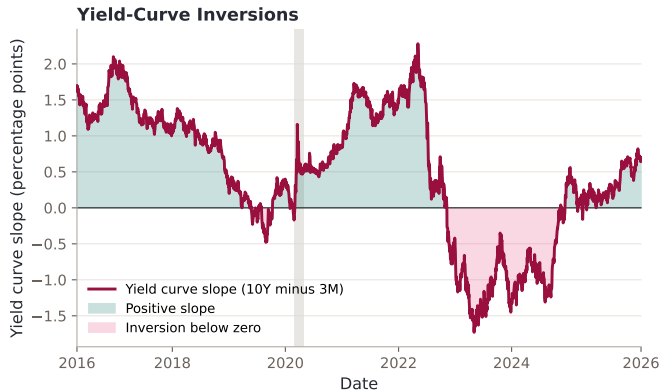
What to look at.

- Three lines (DGS10, DGS2, DTB3), levels, monthly month-end
- A legend distinguishes the three series (10Y, 2Y, 3M)
- The 2020 collapse to zero and the 2022–2023 rise

Source. FRED DGS10, DGS2, DTB3; 2015-01 to 2025-12.

Built by: `make_fred_market_figures.py`.

WORKED EXAMPLE: YIELD-CURVE INVERSION



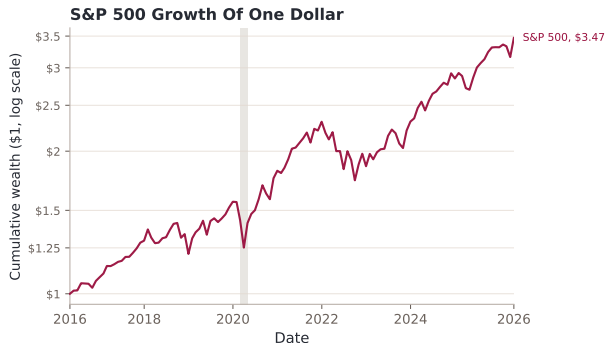
Question. When was the U.S. yield curve inverted, and how deeply?

What to look at.

- Single series (10Y minus 2Y) shaded above and below zero
- Positive (normal) regions in one colour, inverted regions in the emphasis colour
- Count distinct inversion episodes in the sample
- Read the depth of the deepest inversion off the y-axis

Source. FRED T10Y2Y, 2015-01 to 2025-12.

WORKED EXAMPLE: S&P 500 GROWTH OF \$1



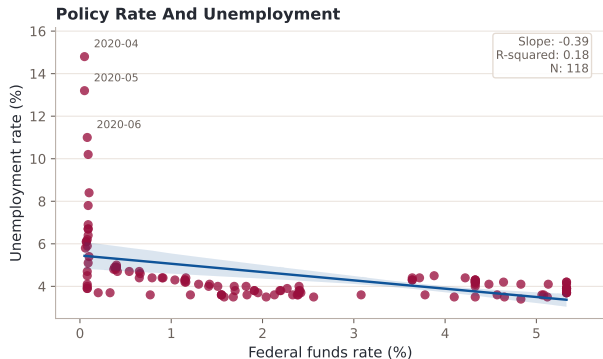
Question. How did \$1 invested in the S&P 500 grow, and what were the drawdowns?

What to look at.

- Y-axis is in dollars but spaced logarithmically
- Equal vertical distance equals equal percentage change
- Read the rightmost tick: that is the ending wealth, not the cumulative return

Source. FRED SP500, 2015-01 to 2025-12.

WORKED EXAMPLE: FEDERAL FUNDS RATE VS UNEMPLOYMENT



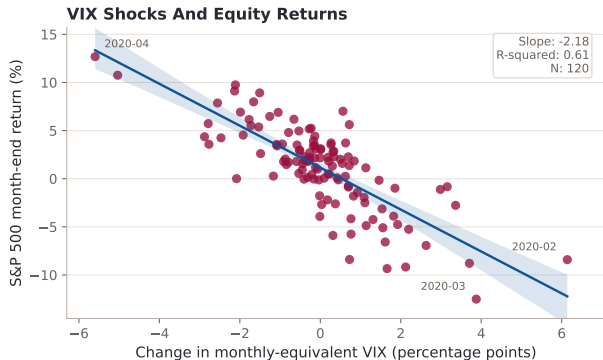
Question. What is the empirical relationship between the policy rate and unemployment?

What to look at.

- Each point is one month, 2015–2025
- Look for clusters that signal a regime (e.g., the 2020–2021 zero-rate cluster)
- OLS line is a **visual aid only**: no slope test, no forecast

Source. FRED FEDFUNDS and UNRATE.

WORKED EXAMPLE: EQUITY RETURNS VS THE VIX



Question. How do equity returns co-move with changes in implied volatility?

What to look at.

- Y-axis: monthly S&P 500 return. X-axis: monthly change in the VIX
- OLS line is a **visual aid only**

Source. FRED SP500 and VIXCLS, 2015-01 to 2025-12.

WRITING THE NARRATIVE

Once the five figures are built, write a six-paragraph narrative that ties them into a story. The structure mirrors the Australian narrative:

1. Opening paragraph — a sentence per figure, naming the question each answers
2. Paragraph on monetary policy and Treasury rates (figures 1 and 2)
3. Paragraph on equity returns and wealth growth (figure 3)
4. Paragraph on the policy-rate / labour-market relationship (figure 4)
5. Paragraph on the equity-volatility relationship (figure 5)
6. Closing paragraph — what does the U.S. economy look like in late 2025?

Five well-built figures and six paragraphs of written narrative — roughly 600 words plus the figures, with no modelling or forecasting required.

DEPLOY THE WEEK 2 APP

THIS WEEK THE APP IS DEPLOYED

If you already completed the Week 0 Streamlit rehearsal, use this part as a quick checklist. If you did not, do it now.

The app for this week already exists:

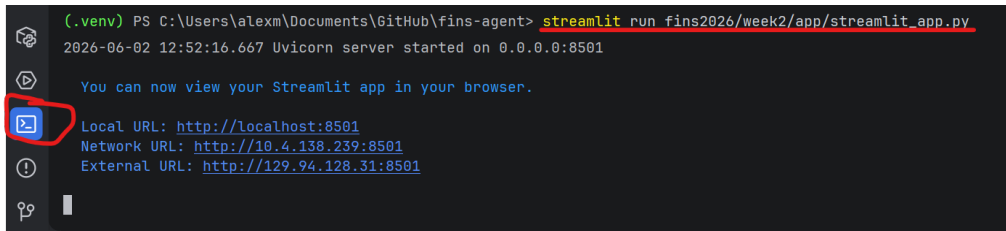
- It is the **U.S. Month-End Market and Macro Monitor**
- The file is `fins2026/week2/app/streamlit_app.py`
- Your job is to publish that working app cleanly, not redesign it

Week 2 is the publish step: run the app locally, let your agent prepare the clean repo, then deploy it.

STEP 1 — RUN THE EXISTING APP LOCALLY

In PyCharm, open the **Terminal** tab (the icon on the left edge, circled below), then paste this command and press Enter:

```
streamlit run fins2026/week2/app/streamlit_app.py
```



Open the **Local URL** (<http://localhost:8501>) in your browser to confirm the app runs before you publish it.

STEP 2 — USE YOUR AGENT TO PREPARE THE REPO

With the app running locally, paste this prompt into your agent:

```
Use docs/apps/streamlit/student-quickstart.md  
to deploy the Week 2 Streamlit app.
```

```
Keep the app unchanged.
```

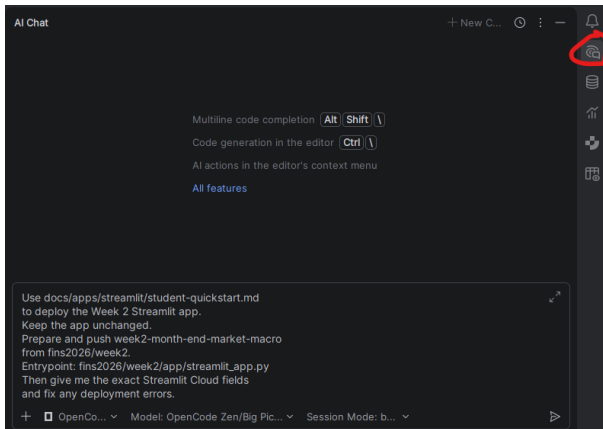
```
Prepare and push week2-month-end-market-macro  
from fins2026/week2.
```

```
Entrypoint: fins2026/week2/app/streamlit_app.py
```

```
Then give me the exact Streamlit Cloud fields  
and fix any deployment errors.
```

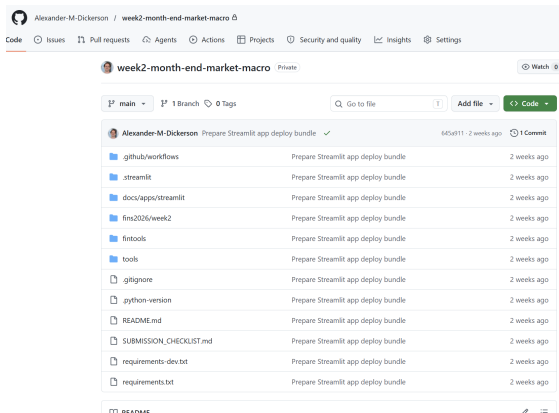
STEP 2 — WHERE TO PASTE THE PROMPT

Open the **AI Chat** panel in PyCharm (the icon on the right-hand toolbar, circled below), then paste the prompt into the input box at the bottom and send it:



WHAT YOUR REPOSITORY SHOULD LOOK LIKE ON GITHUB

After the agent has prepared and pushed the repository, open it on GitHub and confirm it looks like this:



`fins2026/week2` holds the app, `requirements.txt` lists the packages Streamlit Cloud installs, and the entrypoint is `fins2026/week2/app/streamlit_app.py`.

DEPLOY BY FILLING IN FOUR BOXES

Only do this once the agent has finished setting up the repository and pushed it to GitHub.

On share.streamlit.io, create a new app from GitHub and use:

- **Repository:** your-github-username/week2-month-end-market-macro
- **Branch:** main
- **Main file path:** fins2026/week2/app/streamlit_app.py
- **Python version:** 3.13

Leave secrets empty for Week 2.

- If the repo does not appear, reconnect GitHub under **Linked accounts** or use **Paste GitHub URL**
- After deploy, set **Settings** → **Sharing** → **This app is public and searchable**
- Test the public `streamlit.app` URL in an incognito window

THIS PROCESS IS CENTRAL TO THE PROJECT

We follow this same publish process every week, and it is not just an exercise:

- The project also requires a Streamlit app, deployed the same way.
- Every week, your app must run when it is deployed from your GitHub repository, not only on your own machine.
- The weekly deploy is deliberate practice for the project hand-in: if it works cleanly every week, it will work when you submit the project.

Treat each weekly deploy as a rehearsal for the project: a Streamlit app that runs from GitHub is the standard you are marked against.

SUMMARY

Key points from this lecture:

- A decision-maker reads your figures, not your code or table, and a narrative is a sequence of figures
- Every figure should satisfy the four standards: caption, units, source, sample window
- The Australian macro toolkit is twelve series across monetary, inflation, real-economy, and external-sector groupings
- Reading from RBA or FRED is a `pd.read` call per table or series, with type-checking and lag-checking the next few lines
- Stage 1 is where release dates are mapped: `reference_date`, `release_date`, `observable_month_end`
- The transformation choice follows from the *question*, not from the dataset
- Five well-built figures and the written narrative that ties them together are a complete deliverable — no model required
- The Week 2 app can be published as-is once it runs locally. Deployment is a workflow step, not a new modelling task

QUESTIONS?

REFERENCES I

- Alberto Cairo. *The Functional Art: An Introduction to Information Graphics and Visualization*. New Riders, Berkeley, CA, 2012. ISBN 9780321834737.
- Juraj Hric and Yiping Lin. *Applied Data Science in FinTech: Models, Tools, and Case Studies*. Routledge, London, 2026. ISBN 9781032762579. URL <https://www.routledge.com/Applied-Data-Science-in-FinTech-Models-Tools-and-Case-Studies/Hric-Lin/p/book/9781032762579>. Course-mandated textbook.
- Wes McKinney. *Python for Data Analysis: Data Wrangling with pandas, NumPy, and Jupyter*. O'Reilly Media, 3 edition, 2022. ISBN 9781098104030. URL <https://wesmckinney.com/book/>.
- Cole Nussbaumer Knaflic. *Storytelling with Data: A Data Visualization Guide for Business Professionals*. John Wiley & Sons, Hoboken, NJ, 2015. ISBN 9781119002253.
- Jonathan Schwabish. *Better Data Visualizations: A Guide for Scholars, Researchers, and Wonks*. Columbia University Press, New York, 2021. ISBN 9780231193115. URL <https://cup.columbia.edu/book/better-data-visualizations/9780231193115>.
- Edward Segel and Jeffrey Heer. Narrative visualization: Telling stories with data. *IEEE Transactions on Visualization and Computer Graphics*, 2010. doi: 10.1109/TVCG.2010.179. URL <https://idl.uw.edu/papers/narrative>.
- Edward R. Tufte. *The Visual Display of Quantitative Information*. Graphics Press LLC, 2 edition, 2001. URL <https://www.edwardtufte.com/book/the-visual-display-of-quantitative-information/>.
- Claus O. Wilke. *Fundamentals of Data Visualization: A Primer on Making Informative and Compelling Figures*. O'Reilly Media, Sebastopol, CA, 2019. ISBN 9781492031086. URL <https://clauswilke.com/dataviz/>.